

CSA17 - Analytical Problem-Solving Report

Artificial Intelligence - Assessment Tool 1

Name: Y. Roshitha

Reg No: 192424400

Q1. Water Jug Problem (4-gallon & 3-gallon)

Problem Explanation

Two unmarked jugs of capacity 4 gallons and 3 gallons are available along with a pump. The objective is to measure out exactly 2 gallons using only fill, empty, and pour operations.

Approach

Rather than hand-tracing a fixed sequence of moves, this solution treats every reachable (Jug-A, Jug-B) pair as a state and performs a Breadth-First Search over the state graph, guaranteeing the shortest sequence of fill/empty/pour actions that reaches a state where Jug-A holds 2 gallons.

Python Code

```
def water_jug_bfs(cap_a=4, cap_b=3, target=2):
    start = (0, 0)
    visited = {start}
    parent = {start: None}
    queue = deque([start])

    def next_states(state):
        a, b = state
        return [
            (cap_a, b), (a, cap_b), (0, b), (a, 0),
            (a - min(a, cap_b - b), b + min(a, cap_b - b)),
            (a + min(b, cap_a - a), b - min(b, cap_a - a)),
        ]

    goal_state = None
    while queue:
        state = queue.popleft()
        if state[0] == target:
            goal_state = state
            break
        for nxt in next_states(state):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = state
                queue.append(nxt)

    # reconstruct path via parent pointers ...
```

Output

Final state reached: Jug-A = 2 gallons, Jug-B = 3 gallons, in 6 BFS steps from the empty start state (see Output/A1_Output_AI.txt for the full trace).

Q2. Mars Rover Intelligent Agent

Concept Explanation

The Mars rover is modelled as a model-based, goal-driven agent operating in a partially observable, dynamic, and hazardous environment, tracking an internal record of samples already analysed to decide its next action.

Percepts

Camera feed, spectrometer reading, soil probe data, ambient temperature, terrain type, obstacle-detection flag, dust-storm alert.

Actions

Drive forward/backward, rotate left/right, photograph surface, extract core sample, run spectral analysis, uplink data to orbiter.

Performance Measures

Samples analysed, remaining battery reserve, hazard avoidance, and whether the mission-success threshold (≥ 2 samples analysed with adequate battery margin) is met.

Suitable Agent Architecture

A model-based agent with a goal component: an internal model tracks progress toward the sampling goal, while a thin reactive layer handles immediate obstacle response.

Python Code

```
class MarsRoverAgent:
    SAFE_ENERGY_THRESHOLD = 20

    def __init__(self):
        self.battery = 100
        self.samples_analysed = 0

    def percepts(self):
        return {"camera_feed": "surface_image", "ambient_temp_c": -63, ...}

    def analyse_sample(self):
        self.battery -= 12
        self.samples_analysed += 1

    def performance_measure(self):
        mission_success = (self.samples_analysed >= 2
                           and self.battery >= self.SAFE_ENERGY_THRESHOLD)
        return {"battery_remaining": self.battery,
                "samples_analysed": self.samples_analysed,
                "mission_success": mission_success}
```

Output

After one drive and two sample analyses: battery_remaining = 72, samples_analysed = 2, mission_success = True.

Q3. 8-Queens Problem

Problem Explanation

Eight queens must be placed on a chessboard so that no two queens share a row, column, or diagonal.

Search Components

State: partial column-wise queen placement. Initial state: empty board. Successor function: place a queen in the next column in any row not attacked by existing queens. Goal test: all 8 columns filled with no conflicts. Strategy: backtracking DFS, accelerated with bitmasks for column and diagonal occupancy.

Python Code

```
def solve_queens_bitmask():
    solutions, placement = [], []

    def place(row, cols, diag1, diag2):
        if row == BOARD_SIZE:
            solutions.append(placement[:]); return
        free = ALL_PLACED & ~(cols | diag1 | diag2)
        while free:
            bit = free & (-free)
            col = bit.bit_length() - 1
            placement.append(col)
            place(row + 1, cols | bit, (diag1 | bit) << 1, (diag2 | bit) >> 1)
            placement.pop()
            free &= free - 1

    place(0, 0, 0, 0)
    return solutions
```

Output

Total solutions found: 92. Sample solution (row -> column): [0, 4, 7, 5, 2, 6, 1, 3].

Q4. OLA Cab Booking Agent

Problem Explanation

A customer picks a preferred cab type; the agent must select the best available cab and quote a fare.

Agent Type

Goal-based agent. Goal: book the cab that best balances driver rating and estimated arrival time among cabs matching (or, failing that, closest to) the customer's preference.

Python Code

```
def book_best_cab(source, destination, preferred_type, distance_km=12):
    cabs = fetch_nearby_cabs()
    candidates = [c for c in cabs if c.cab_type == preferred_type] or cabs
    best = max(candidates, key=lambda c: c.driver_rating * 10 - c.eta_minutes)
    fare = round(best.per_km_rate * distance_km, 2)
    return f"Cab booked: {best.cab_type} | ETA={best.eta_minutes} min | " \
           f"Rating={best.driver_rating} | Fare={fare} INR"
```

Output

Cab booked: prime | ETA=3 min | Rating=4.9 | Fare=216 INR.

Q5. Uniform Cost Search (UCS) - Logistics Network

Problem Explanation

Find the least-cost path from warehouse S to warehouse G in a weighted delivery-route graph.

Python Code

```
def uniform_cost_search(start, goal):
    frontier = [(0, start)]
    best_cost, parent, explored = {start: 0}, {start: None}, set()

    while frontier:
        cost, node = heapq.heappop(frontier)
        if node == goal:
            path = []
            while node is not None:
                path.append(node); node = parent[node]
            return cost, list(reversed(path))
        if node in explored:
            continue
        explored.add(node)
        for neighbour, step_cost in delivery_network[node]:
            new_cost = cost + step_cost
            if neighbour not in best_cost or new_cost < best_cost[neighbour]:
                best_cost[neighbour] = new_cost
                parent[neighbour] = node
                heapq.heappush(frontier, (new_cost, neighbour))
```

Output

Least-cost path: S -> A -> C -> G. Total cost: 4.